```
{"title"=>"First article!", "text"=>"This is my first article."}
```

This action is now displaying the parameters for the article that are coming in from the form. However, this isn't really all that helpful. Yes, you can see the parameters but nothing in particular is being done with them.

5.4 Creating the Article model

Models in Rails use a singular name, and their corresponding database tables use a plural name. Rails provides a generator for creating models, which most Rails developers tend to use when creating new models. To create the new model, run this command in your terminal:

```
$ bin/rails generate model Article title:string text:text
```

With that command we told Rails that we want a `Article` model, together with a *title* attribute of type string, and a *text* attribute of type text. Those attributes are automatically added to the `articles` table in the database and mapped to the `Article` model.

Rails responded by creating a bunch of files. For now, we're only interested in `app/models/article.rb` and `db/migrate/20140120191729_create_articles.rb` (your name could be a bit different). The latter is responsible for creating the database structure, which is what we'll look at next.

Active Record is smart enough to automatically map column names to model attributes, which means you don't have to declare attributes inside Rails models, as that will be done automatically by Active Record.

5.5 Running a Migration

As we've just seen, `rails generate model` created a *database migration* file inside the `db/migrate`directory. Migrations are Ruby classes that are designed to make it simple to create and modify database tables. Rails uses rake commands to run migrations, and it's possible to undo a migration after it's been applied to your database. Migration filenames include a timestamp to ensure that they're processed in the order that they were created.

If you look in the `db/migrate/YYYYMMDDHHMMSS_create_articles.rb` file (remember, yours will have a slightly different name), here's what you'll find:

```
class CreateArticles < ActiveRecord::Migration
  def change
    create_table :articles do |t|
      t.string :title
      t.text :text

      t.timestamps null: false
    end
  end
end
```

The above migration creates a method named `change` which will be called when you run this migration. The action defined in this method is also reversible, which means Rails knows how to reverse the change made by this migration, in case you want to reverse it later. When you run this migration it will create an `articles` table with one string column and a text column. It also creates two timestamp fields to allow Rails to track article creation and update times.

For more information about migrations, refer to Rails Database Migrations.

At this point, you can use a rake command to run the migration:

```
$ bin/rake db:migrate
```

Rails will execute this migration command and tell you it created the Articles table.